CS 342: Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

Image by kirkh.deviantart..com. Credit for the slides goes to many people including James Hays, Danny Alexander, Aaron Bobick, and many others. The instructor doesn't claim credit for any slides.

Shanti Mantra

Lyrics in Sanskrit	Meaning in English
ॐ सह नाववतु।	Om, May we all be protected
सह नौ भुनक्तु।	May we all be nourished
सह वीर्यं करवावहै।	May we work together with great energy
तेजस्वि नावधीतमस्तु	May our intelect be sharpened (may our study be effective)
मा विद्विषावहै।	Let there be no Animosity amongst us
ॐ शान्तिः शान्तिः शान्तिः ॥	Om, peace (in me), peace (in nature), peace (in divine forces)

Reference Books

- Computer Vision:
  - E. R. Davies
  - Codeword: Davies

- OpenCV-Python Tutorials Documentation
  - A very useful and simple way to learn Computer Vision
  - Codeword: Tutorial

Study

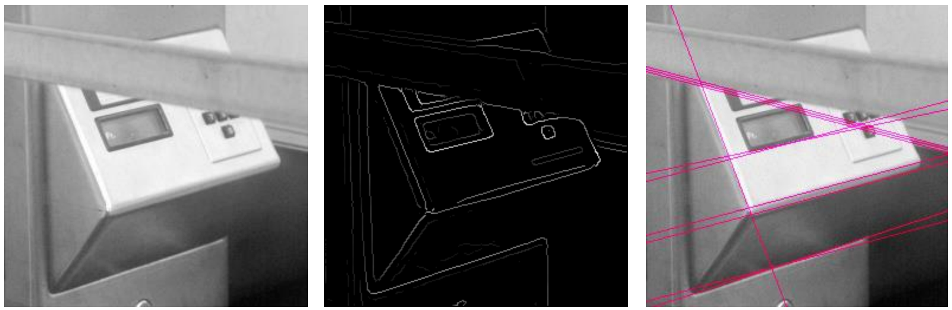
- For This Week:
  - DIP: Global Processing Using the Hough Transform (page: 737-741)
  - Davies:
    - Section 11.1, 11.2, 11.3
    - Section 12.1, 12.2, 12.3
    - Section 13.1, 13.2
  - OpenCV Tutorial:
    - 1.4.13 Hough Line Transform (pdf or link)
    - 1.4.14 Hough Circle Transform

Last time

- Canny Edge Detector
- Today:
  - Hough Transform

Now: Fitting

- Want to associate a model with observed features



[Fig from Marszalek & Schmid, 2007]

For example, the model could be a line, a circle, or an arbitrary shape.

Parametric Model

- A parametric model can represent a class of instances where each is defined by a value of the parameters.
- Examples include lines, or circles, or even a parameterized template

Fitting: Main idea

- Choose a parametric model to represent a set of features

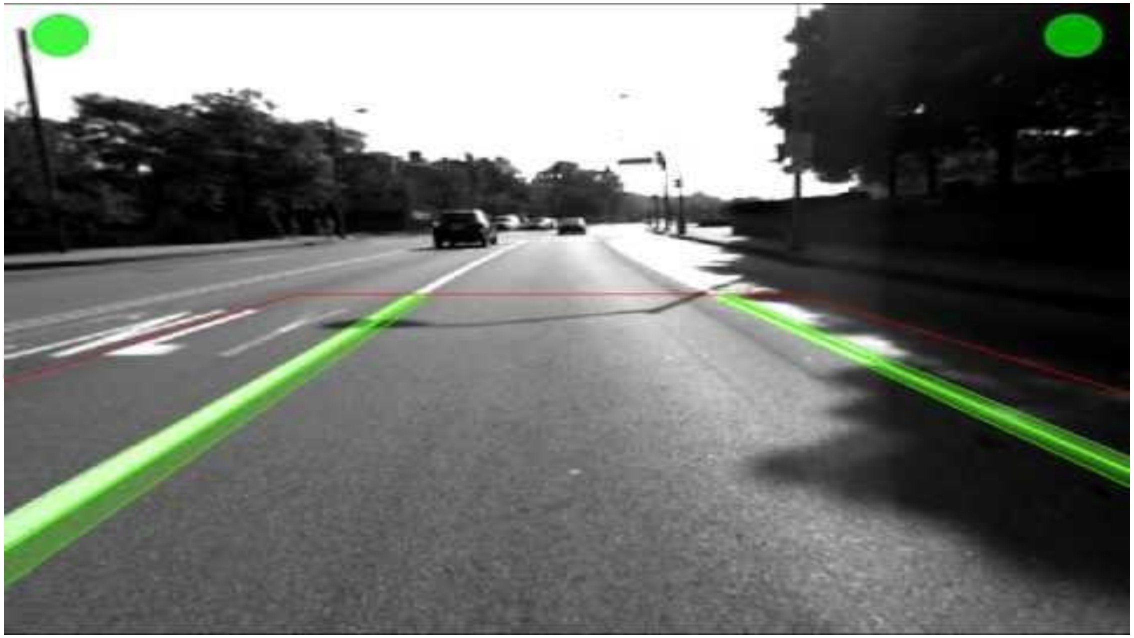
- Membership criterion is not local
  - Can’t tell whether a point belongs to a given model just by looking at that point

- Computational complexity is important
  - It is infeasible to examine every possible set of parameters

Slide credit: L. Lazebnik

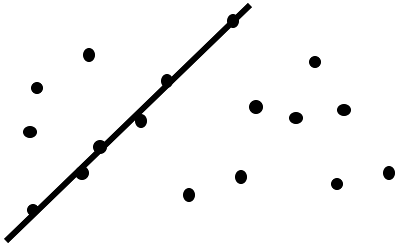
Case study: Line fitting

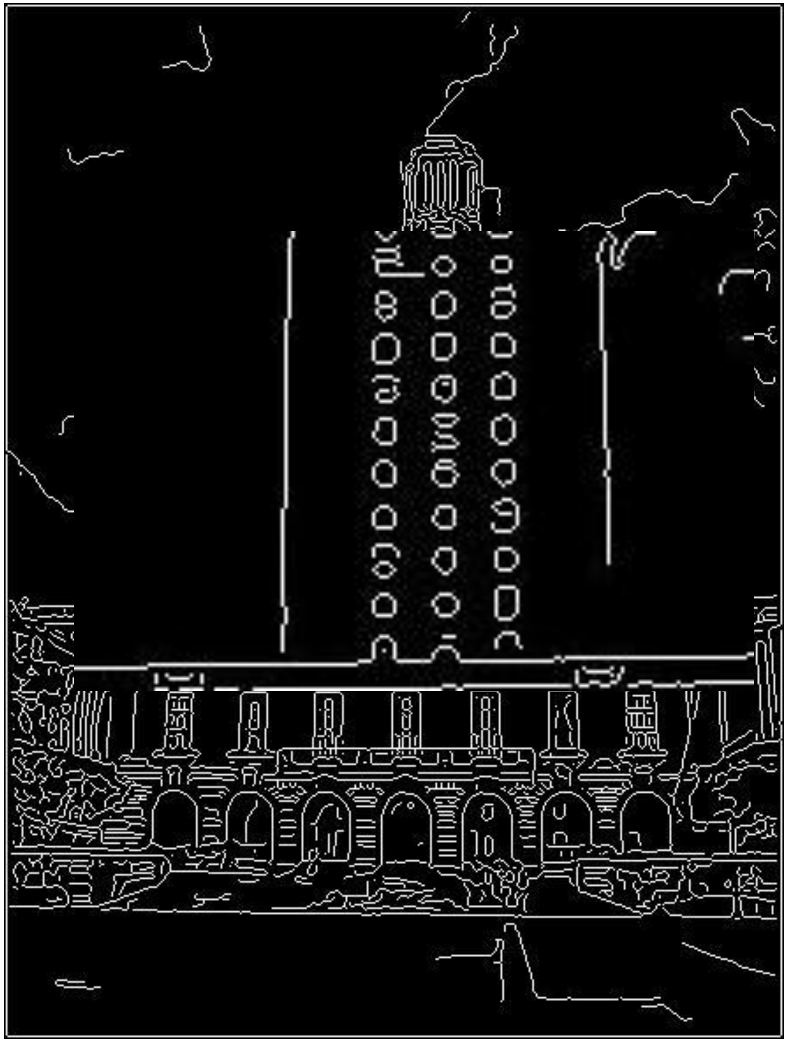
- Why fit lines?
  Many objects characterized by presence of straight lines



- Wait, why aren’t we done just by running edge detection?

Difficulty of line fitting





- **Extra** edge points (clutter), multiple models:
  - which points go with which line, if any?
- Only some parts of each line detected, and some parts are **missing:**
  - how to find a line that bridges missing evidence?
- **Noise** in measured edge points, orientations:
  - how to detect true underlying parameters?

Voting

- It’s not feasible to check all combinations of features by fitting a model to each possible subset.

- **Voting** is a general technique where we let the features *vote for all models that are compatible with it*.
  - Cycle through features, cast votes for model parameters.
  - Look for model parameters that receive a lot of votes.

- Noise & clutter features will cast votes too, *but* typically their votes should be inconsistent with the majority of “good” features.

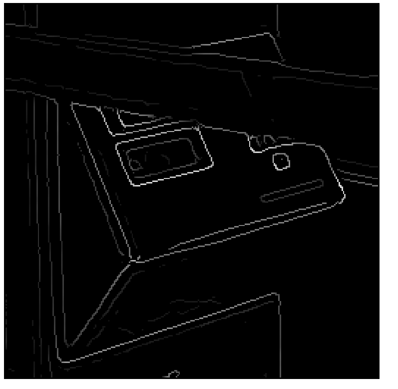
Fitting lines: Hough transform

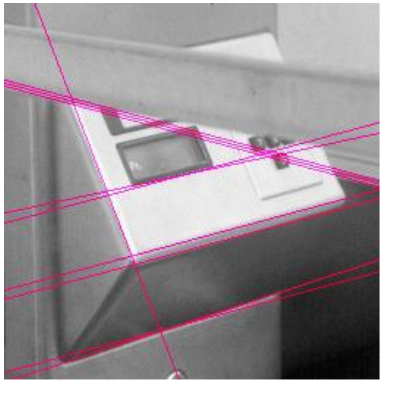
- Given points that belong to a line, what is the line?
- How many lines are there?
- Which points belong to which lines?

- **Hough Transform** is a voting technique that can be used to answer all of these questions.

  Main idea:

  1. Each edge point votes for compatible lines.
  2. Look for lines that get many votes.





Finding lines in an image: Hough space

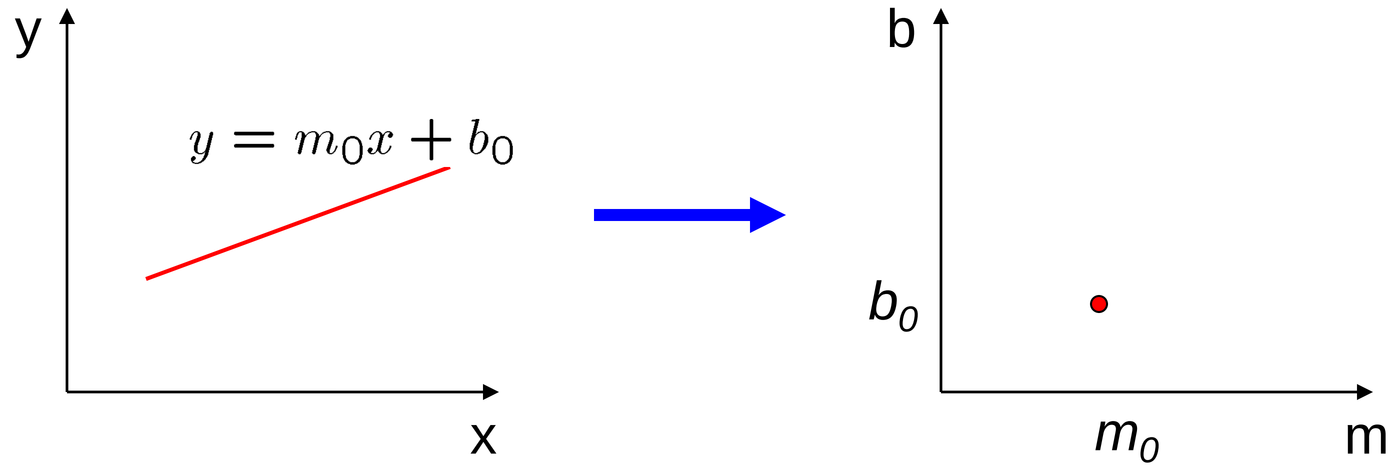


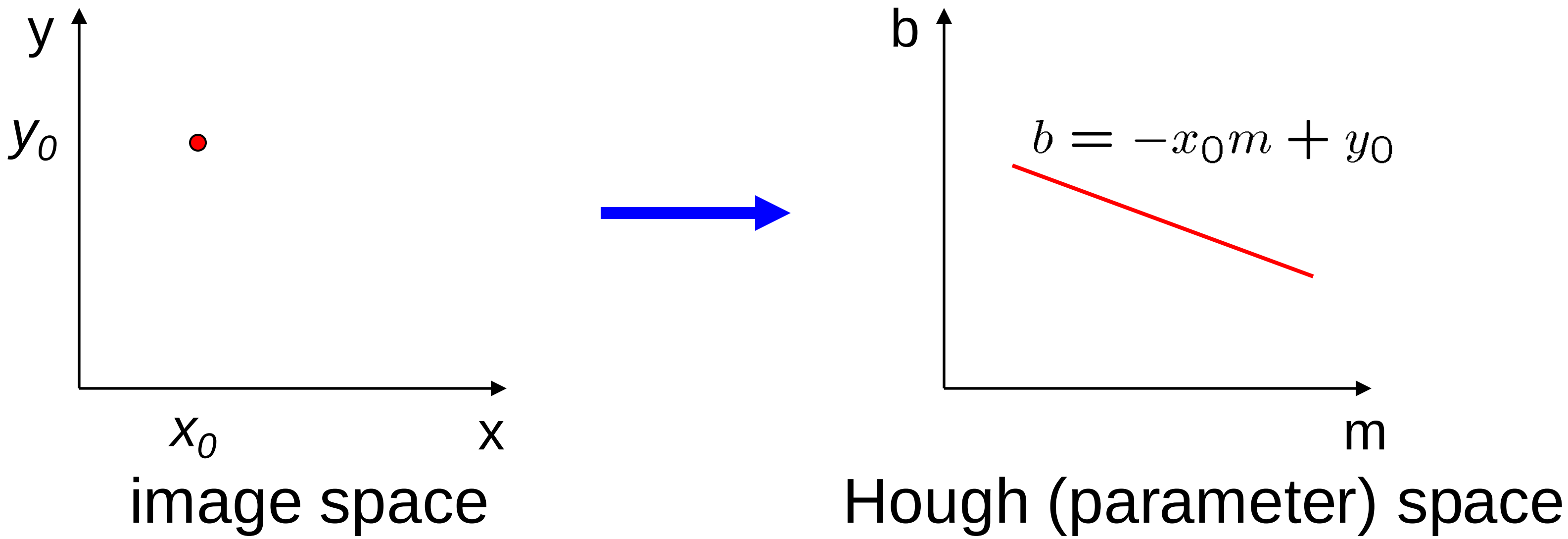
image space | Hough (parameter) space

Connection between image (x,y) and Hough (m,b) spaces

- A line in the image corresponds to a point in Hough space
- To go from image space to Hough space:
  - given a set of points (x,y), find all (m,b) such that y = mx + b

Slide credit: Steve Seitz

Finding lines in an image: Hough space

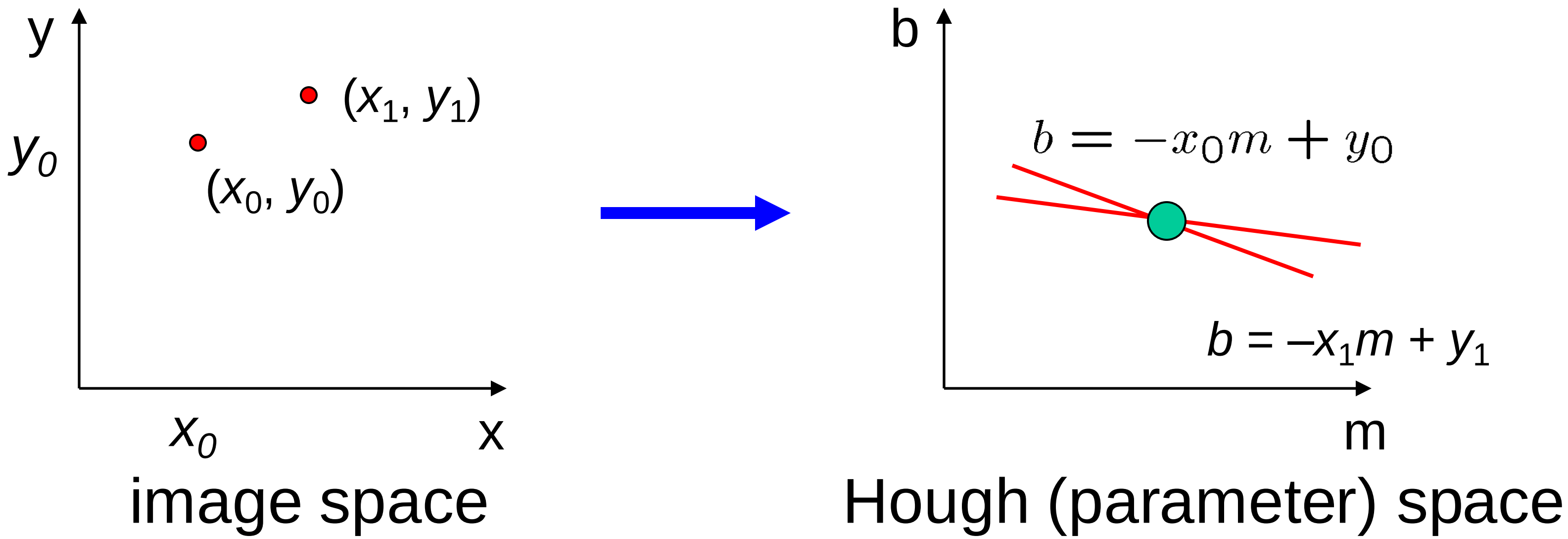


Connection between image (x,y) and Hough (m,b) spaces

- A line in the image corresponds to a point in Hough space
- To go from image space to Hough space:
  - given a set of points (x,y), find all (m,b) such that y = mx + b
- What does a point (x_0, y_0) in the image space map to?
  - Answer: the solutions of $b = -x_0 m + y_0$
  - this is a line in Hough space

Slide credit: Steve Seitz

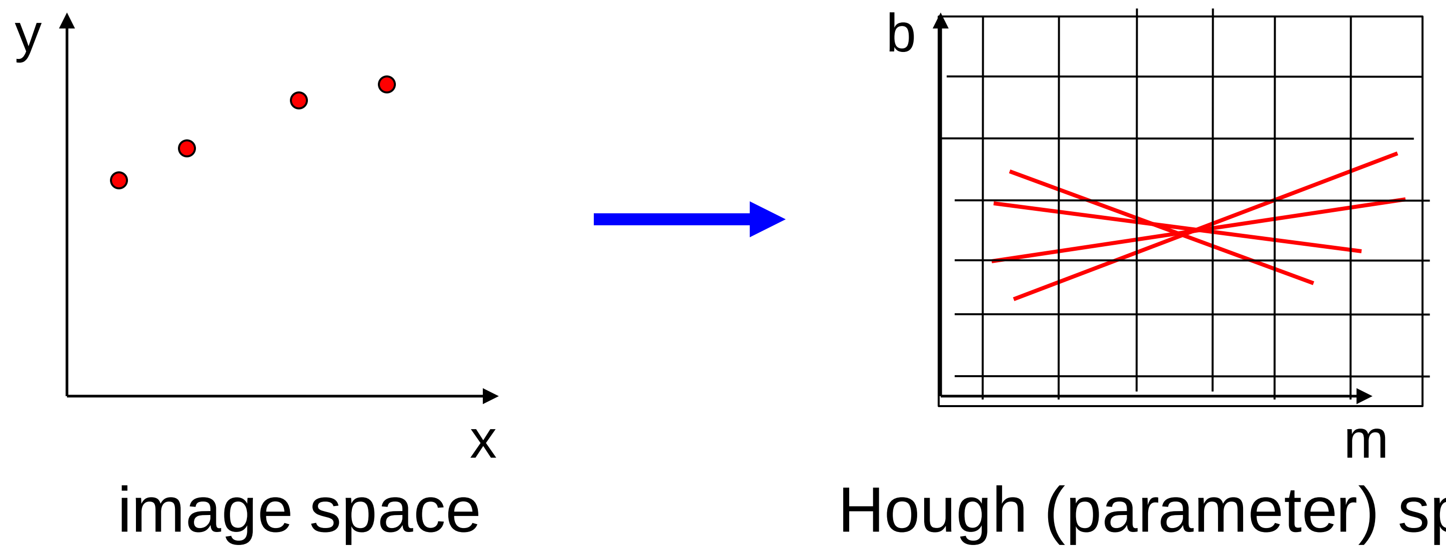
Finding lines in an image: Hough space



What are the line parameters for the line that contains both (x_0, y_0) and (x_1, y_1)?

- It is the intersection of the lines $b = -x_0 m + y_0$ and $b = -x_1 m + y_1$

Finding lines in an image: Hough algorithm

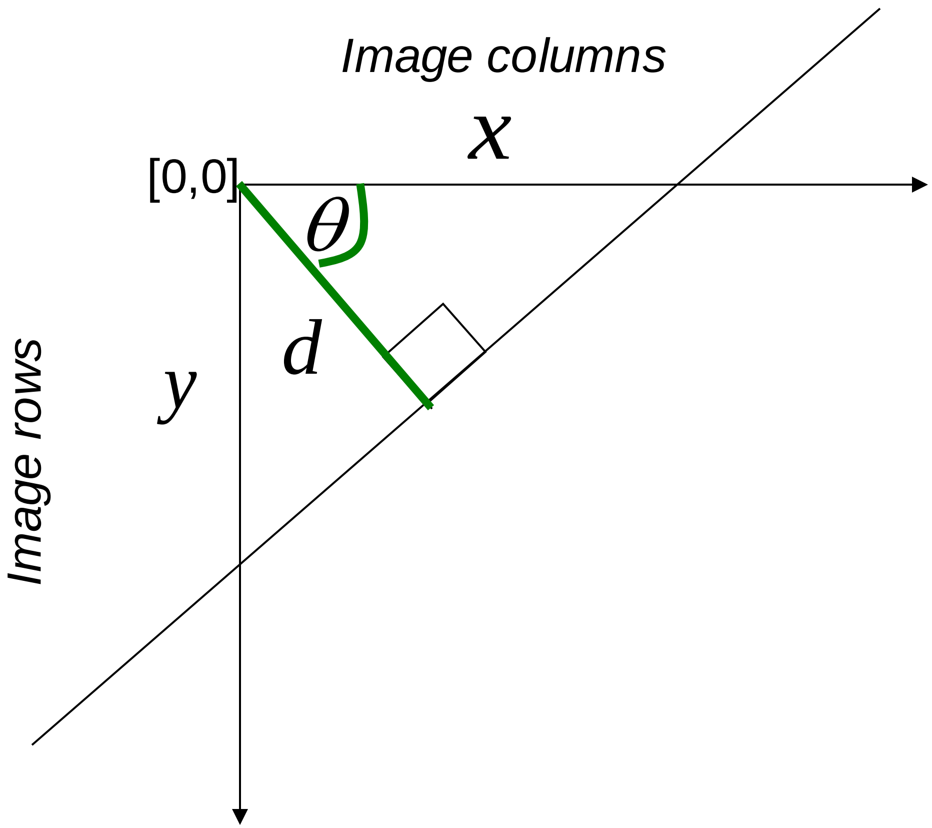


How can we use this to find the most likely parameters (m,b) for the most prominent line in the image space?

- Let each edge point in image space *vote* for a set of possible parameters in Hough space
- Accumulate votes in discrete set of bins*; parameters with the most votes indicate line in image space.

Polar representation for lines

Issues with usual (*m,b*) parameter space: can take on infinite values, undefined for vertical lines.



d : perpendicular distance from line to origin

θ: angle the perpendicular makes with the x-axis

$$x\cos\theta - y\sin\theta = d$$

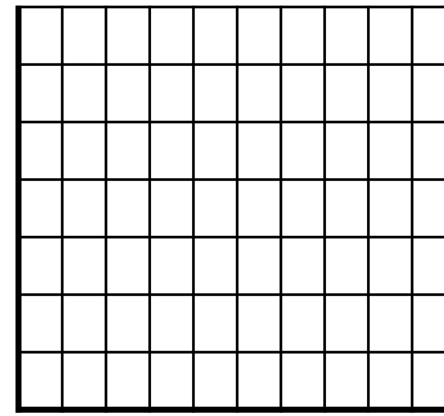
Point in image space → sinusoid segment in Hough space

Hough transform algorithm

Using the polar parameterization:

$$x\cos\theta - y\sin\theta = d$$

H: accumulator array (votes)



d

θ

Basic Hough transform algorithm

1. Initialize H[d, θ]=0
2. for each edge point I[x,y] in the image

   for $\theta = [\theta_{min}$ to $\theta_{max}]$ // some quantization

   $$d = x\cos\theta - y\sin\theta$$

   H[d, θ] += 1
3. Find the value(s) of (d, θ) where H[d, θ] is maximum
4. The detected line in the image is given by $d = x\cos\theta - y\sin\theta$

Time complexity (in terms of number of votes per pt)?

Source: Steve Seitz

Space and Time Complexity

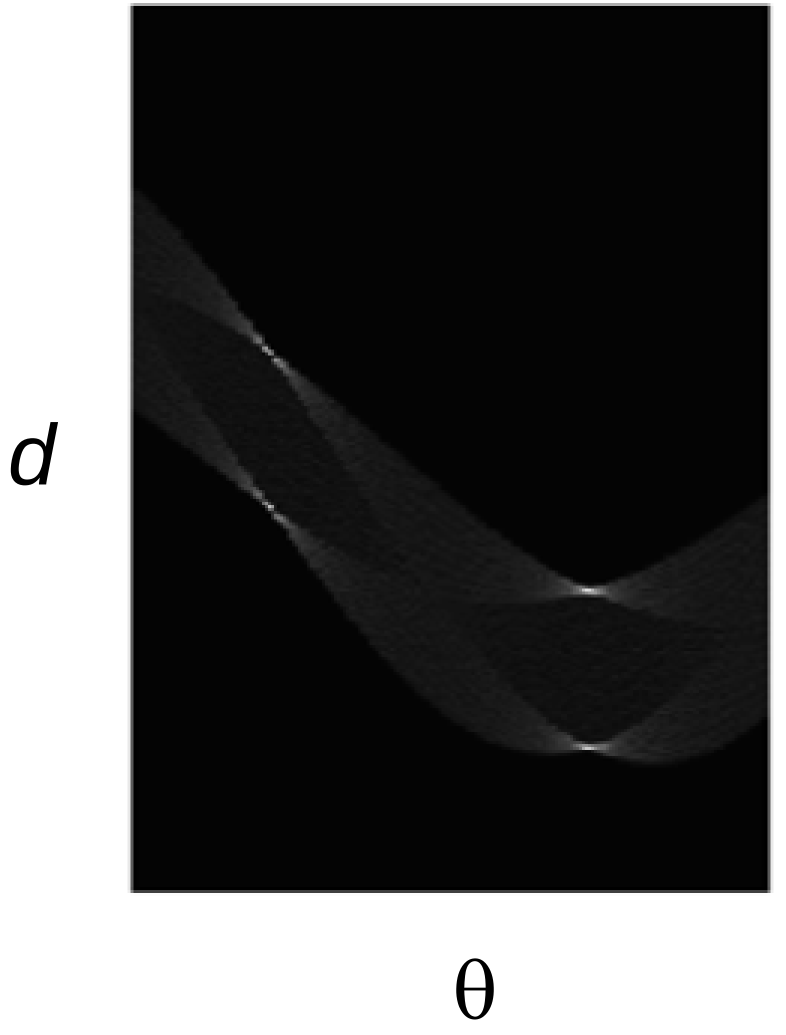
- Space Complexity:
  - k^n bins (n= dimension. In 2D, n = 2)

- Time Complexity:
  - Constant (in terms of number of voting elements i.e., edge points)

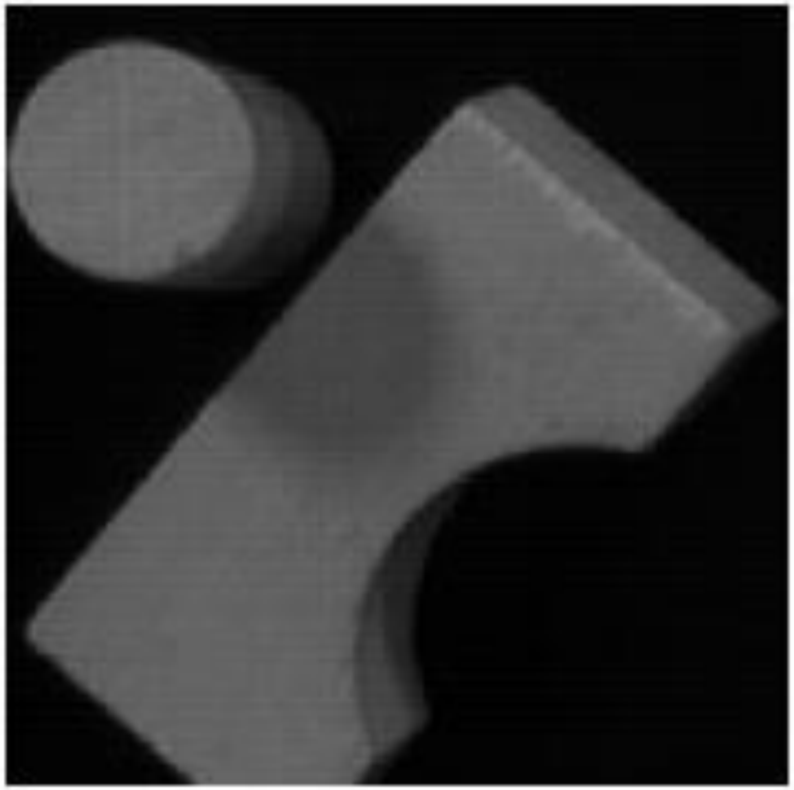

How Hough
Transform
works

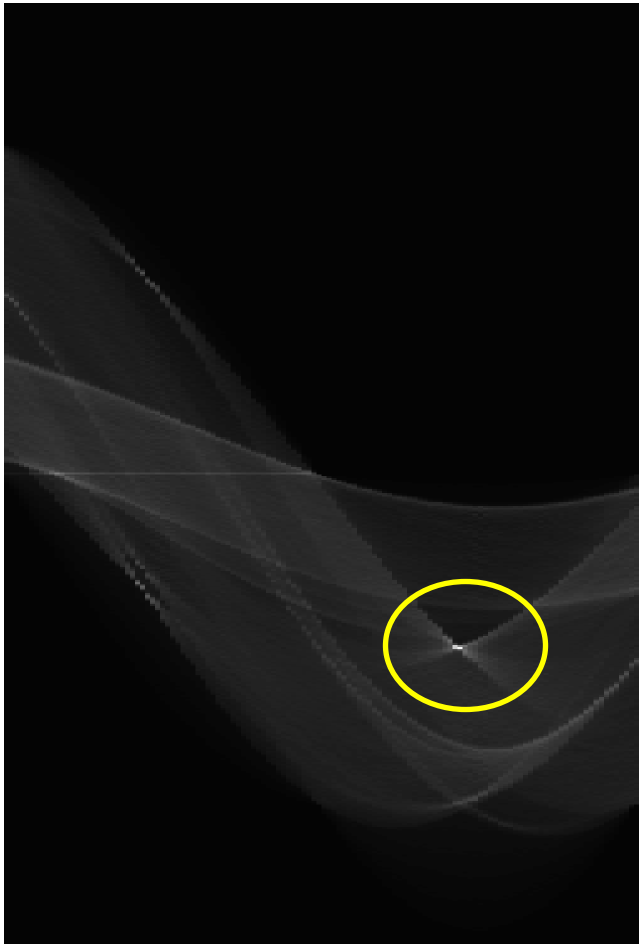
Example: What was the shape?

Square :



Example: Hough transform for straight lines

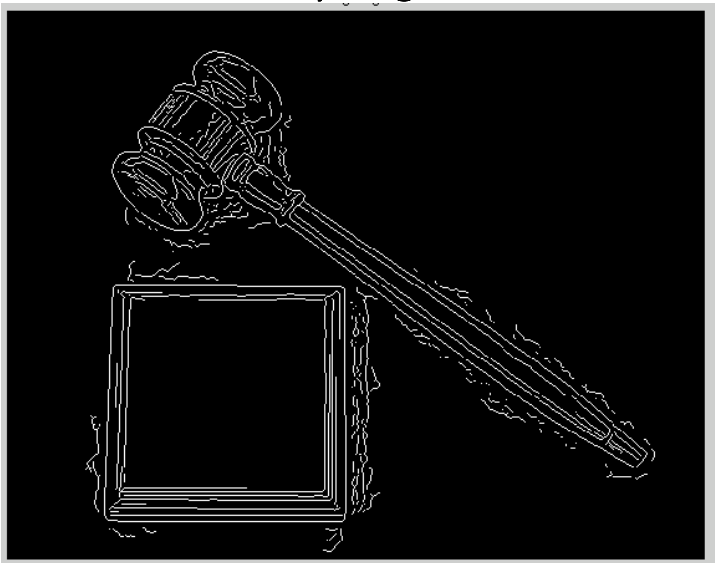




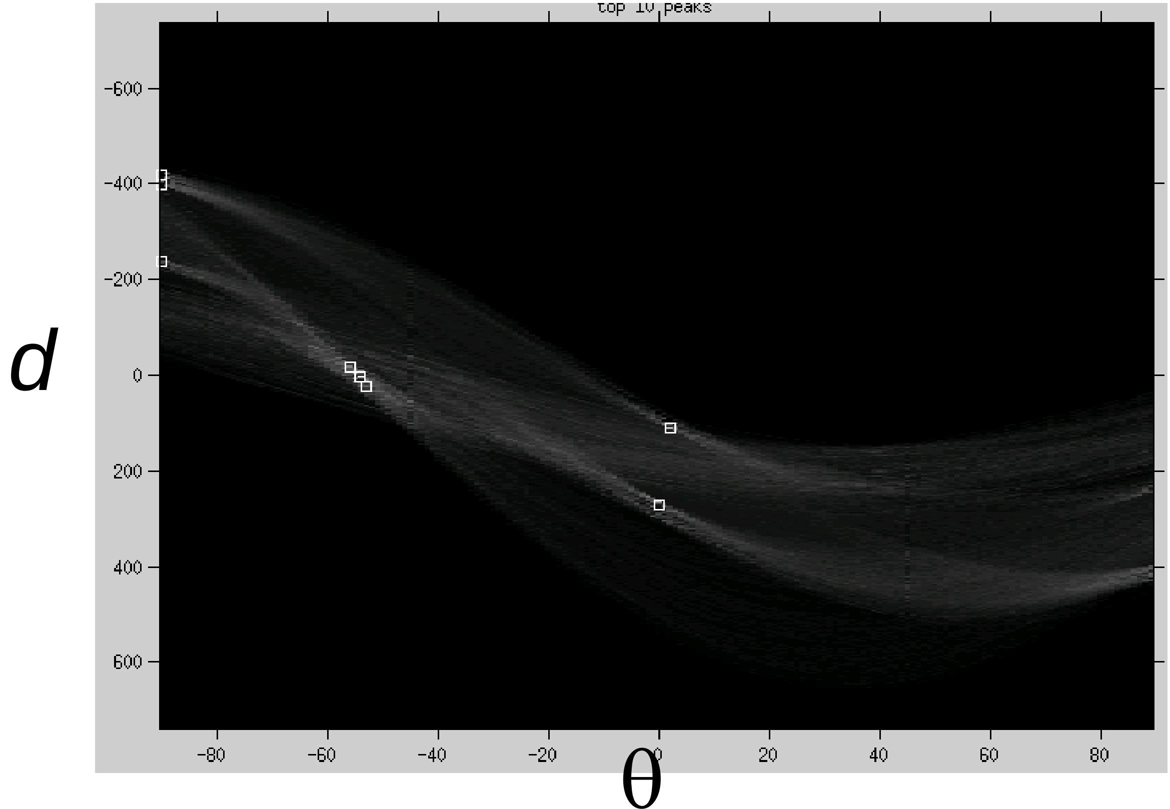
Which line generated this peak?

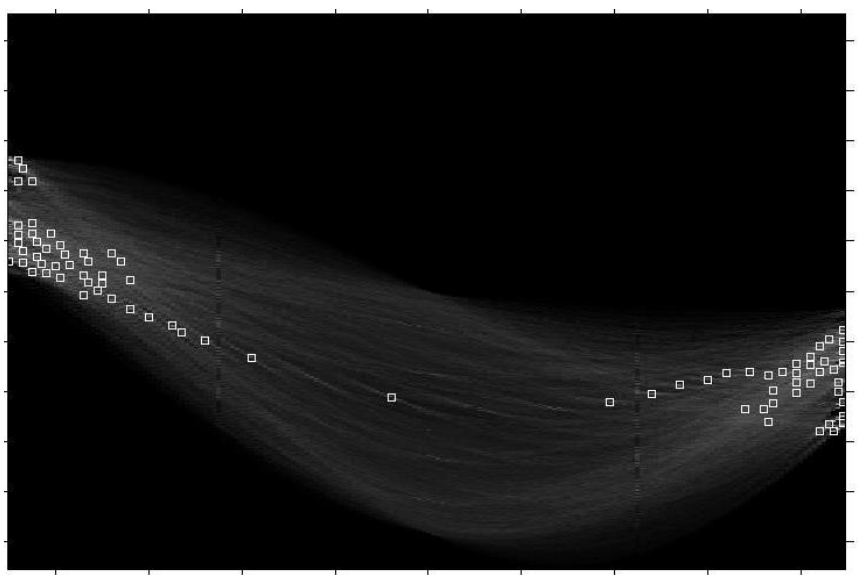
Original image

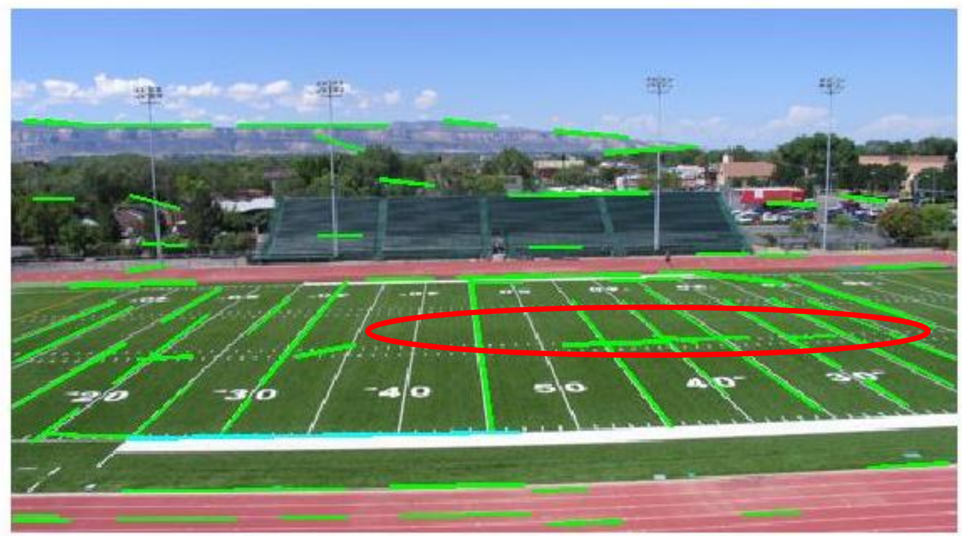
Canny edges



Decode the vote space.

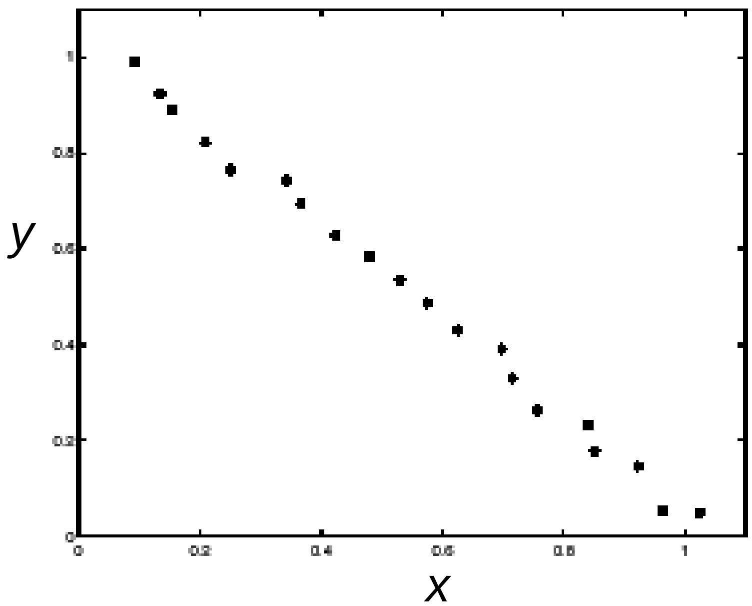






Showing longest segments found

Impact of noise on Hough



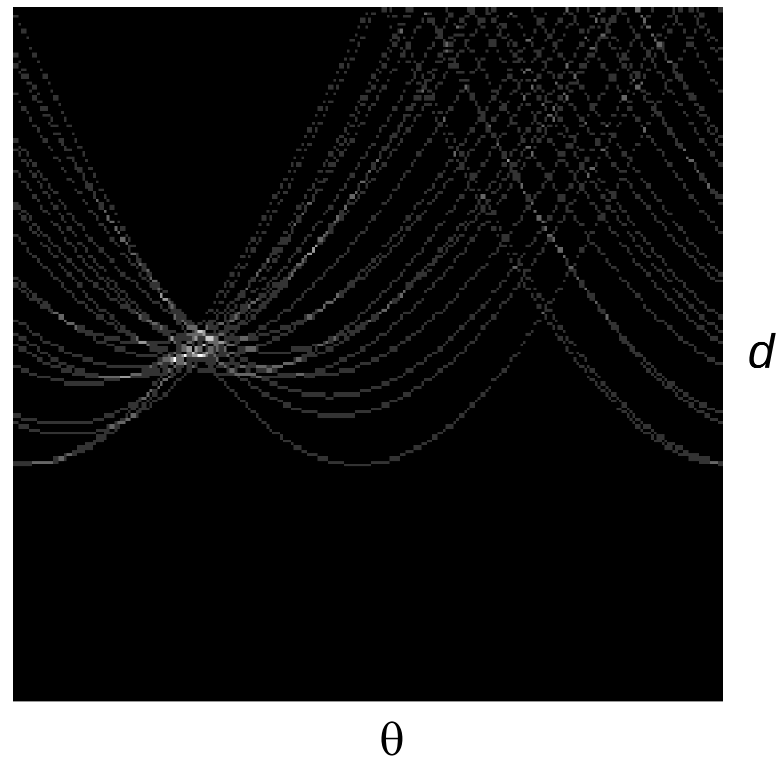


Image space
edge coordinates

Votes

What difficulty does this present for an implementation?

Impact of noise on Hough

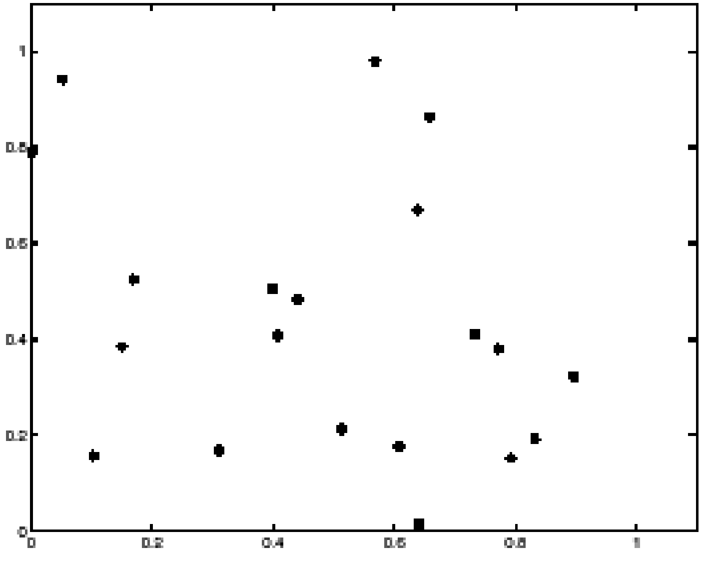
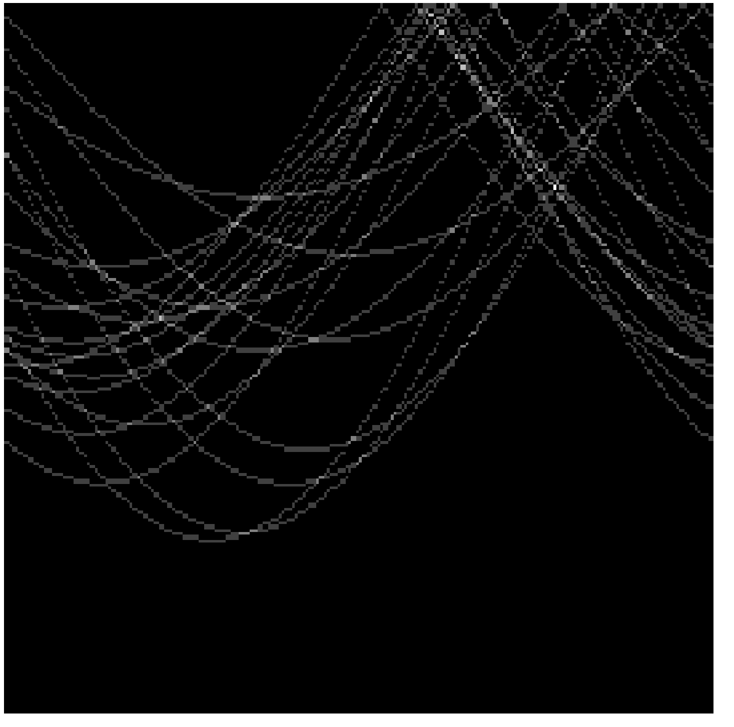


Image space edge coordinates



Votes

Here, everything appears to be “noise”, or random edge points, but we still see peaks in the vote space.

Extensions

Extension 1: Use the image gradient

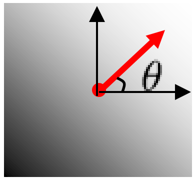
1. same
2. for each edge point I[x,y] in the image

   θ = gradient at (x,y)

   $$d = x\cos\theta - y\sin\theta$$

   H[d, θ] += 1
3. same
4. same

(Reduces degrees of freedom)



$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

Extensions

Extension 1: Use the image gradient

1. same
2. for each edge point I[x,y] in the image

   compute unique (d, θ) based on image gradient at (x,y)

   $H[d, \theta] \mathrel{+}= 1$
3. same
4. same

(Reduces degrees of freedom)

Extension 2

- give more votes for stronger edges (use magnitude of gradient)

Extension 3

- change the sampling of (d, θ) to give more/less resolution

Extension 4

- The same procedure can be used with circles, squares, or any other shape…

Source: Steve Seitz

OpenCV

- Uses cv2.HoughLines

https://docs.opencv.org/3.0-beta/doc/py_tutorials/py_imgproc/py_houghlines/py_houghlines.html

Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For a fixed radius r, unknown gradient direction

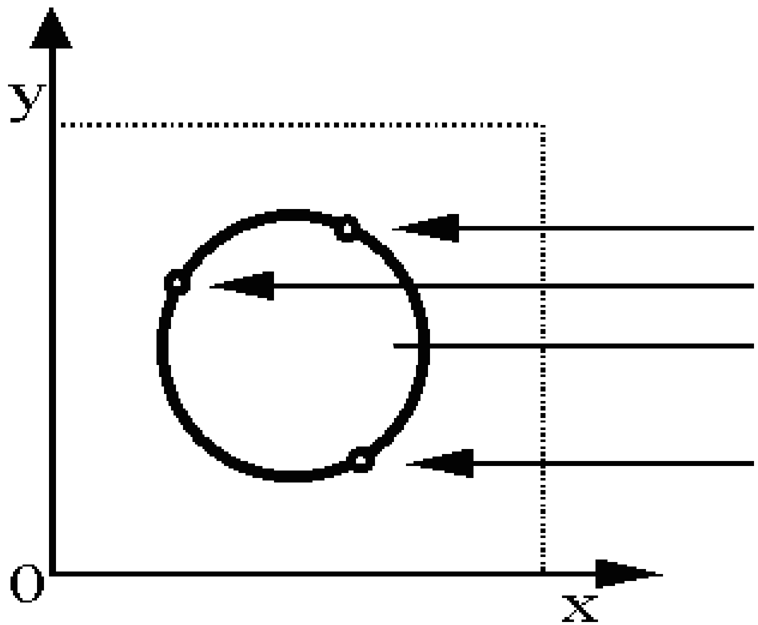
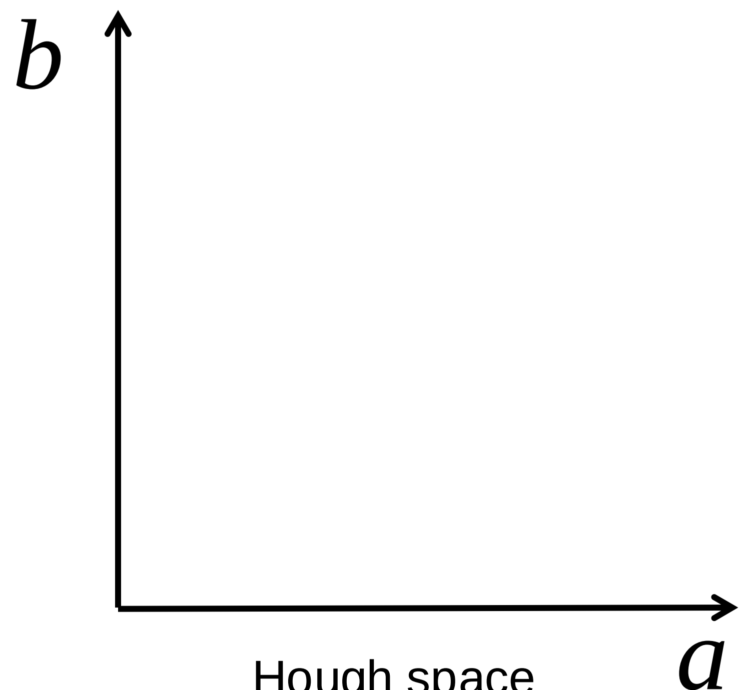


Image space



Hough space

Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For a fixed radius r, unknown gradient direction

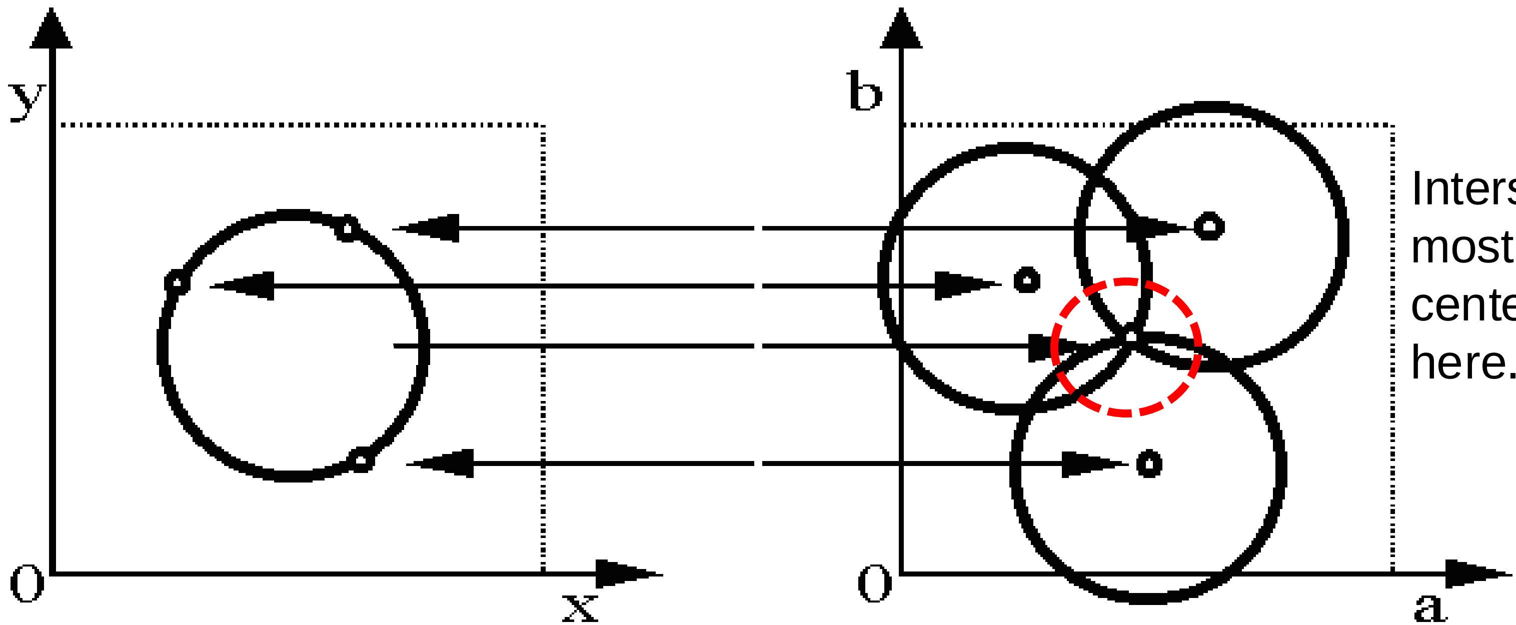


Image space

Hough space

Intersection: most votes for center occur here.

Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For an unknown radius r, unknown gradient direction

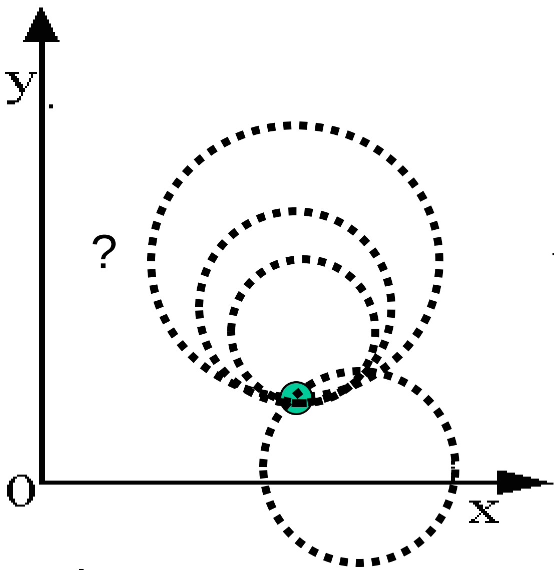
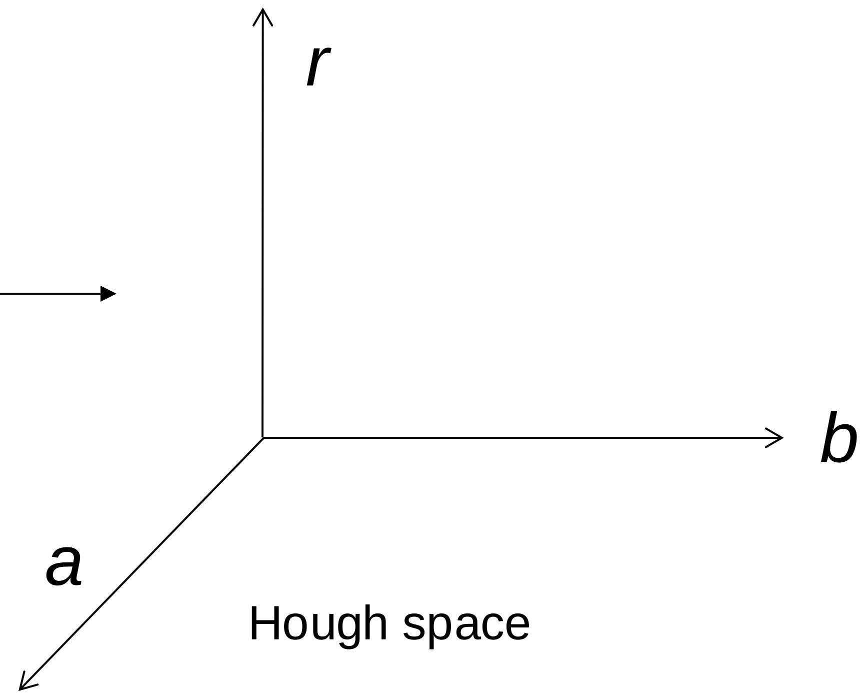


Image space



Hough space

Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For an unknown radius r, unknown gradient direction

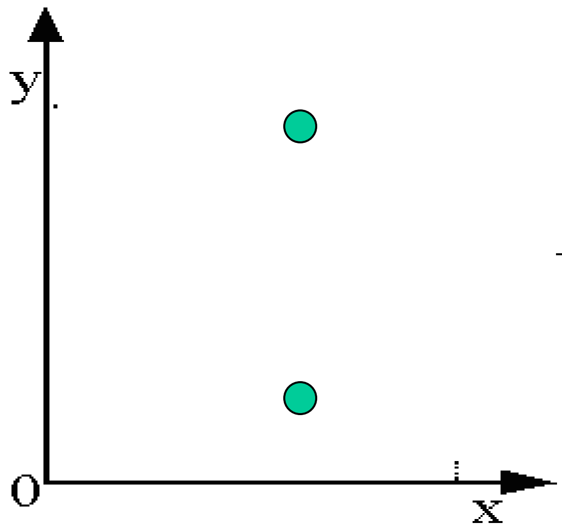
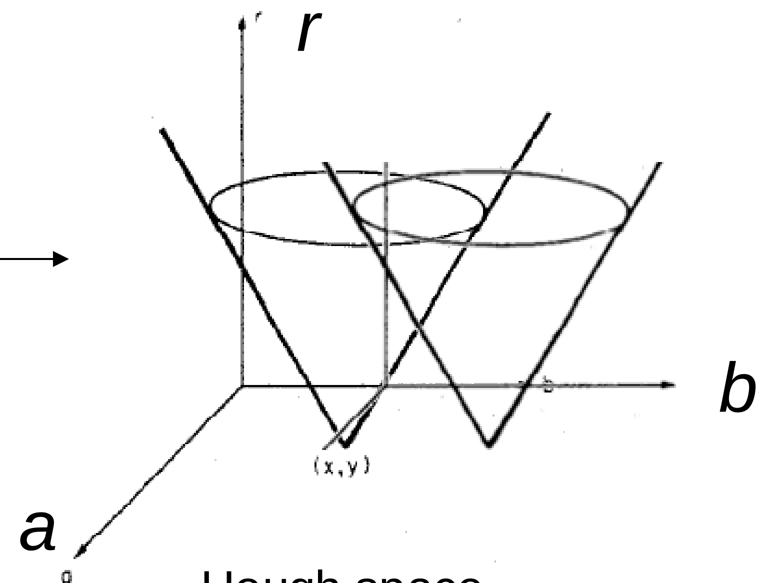


Image space



Hough space

Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For an unknown radius r, **known** gradient direction

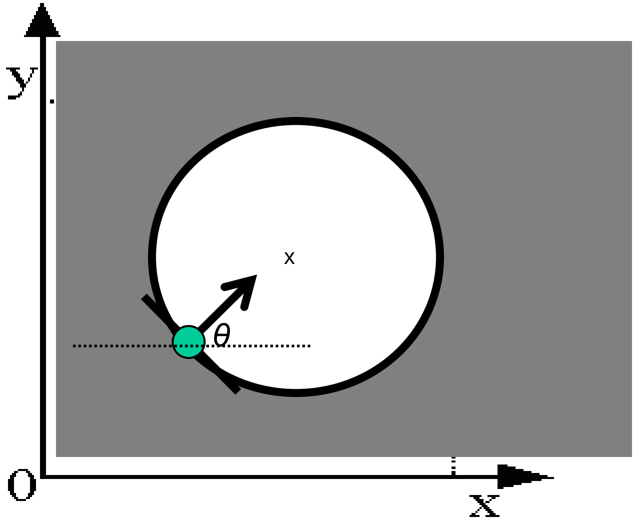
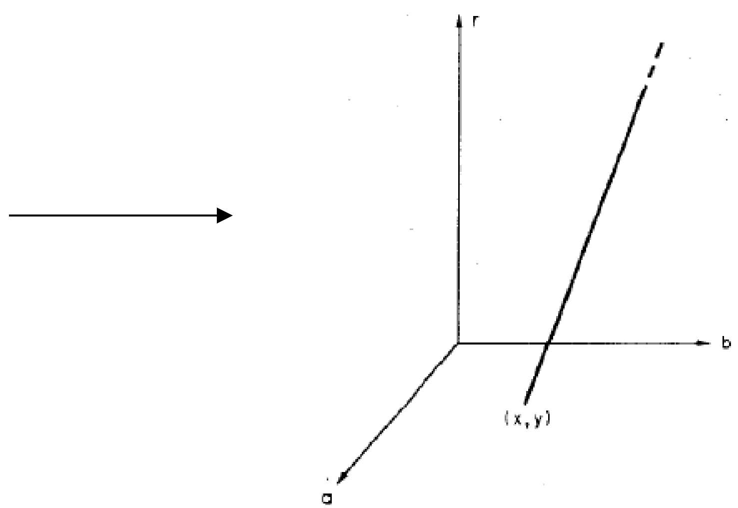


Image space



Hough space

Hough transform for circles

For every edge pixel (x,y) :

For each possible radius value r:

For each possible gradient direction θ:

// or use estimated gradient at (x,y)

$a = x + r\cos(\theta)$ // column

$b = y - r\sin(\theta)$ // row

H[a,b,r] += 1

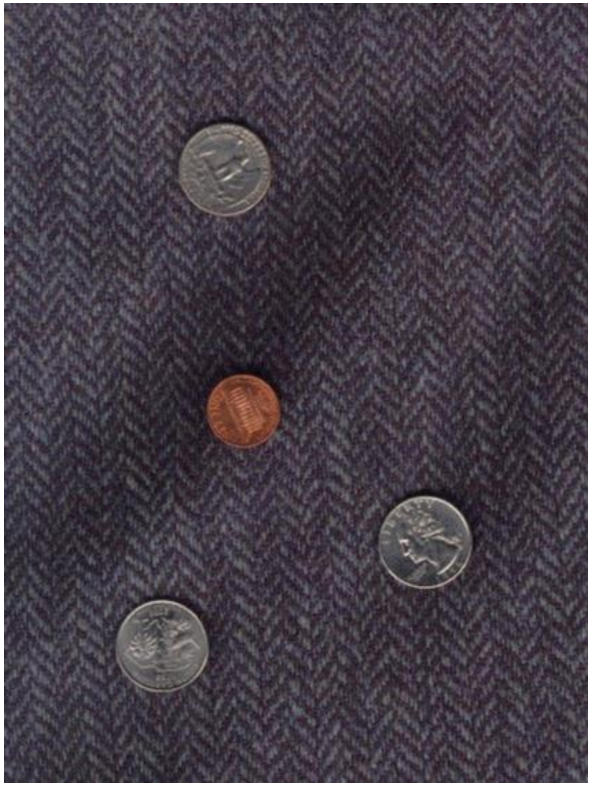
end

end

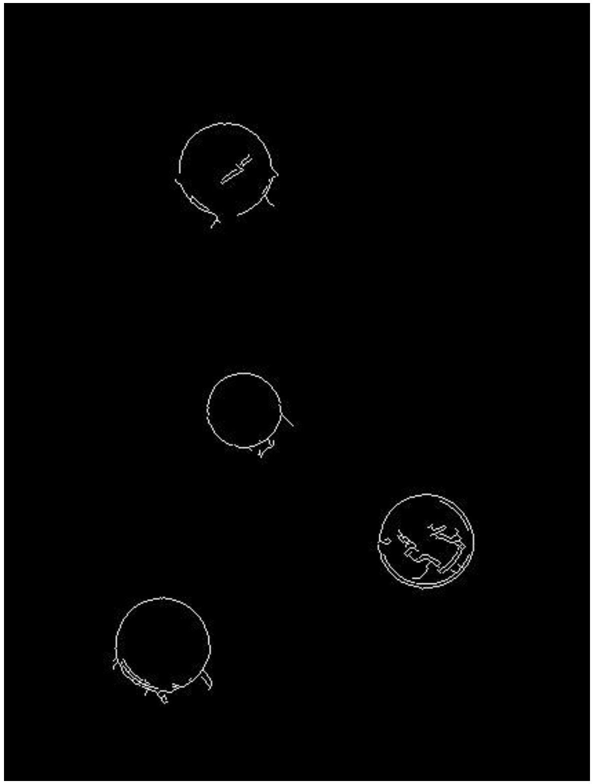
- Check out online demo : http://www.markschulze.net/java/hough/

Example: detecting circles with Hough

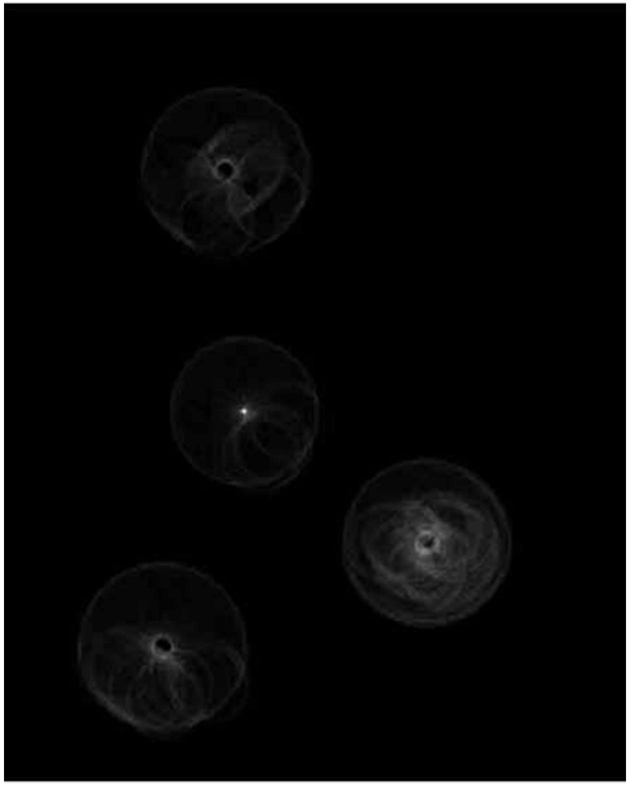
Original



Edges



Votes: Penny

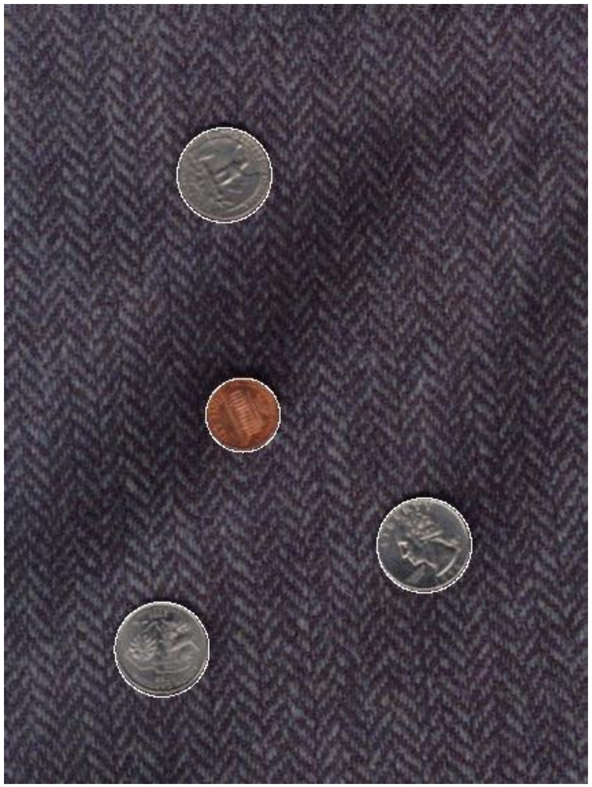


Note: a different Hough transform (with separate accumulators) was used for each circle radius (quarters vs. penny).

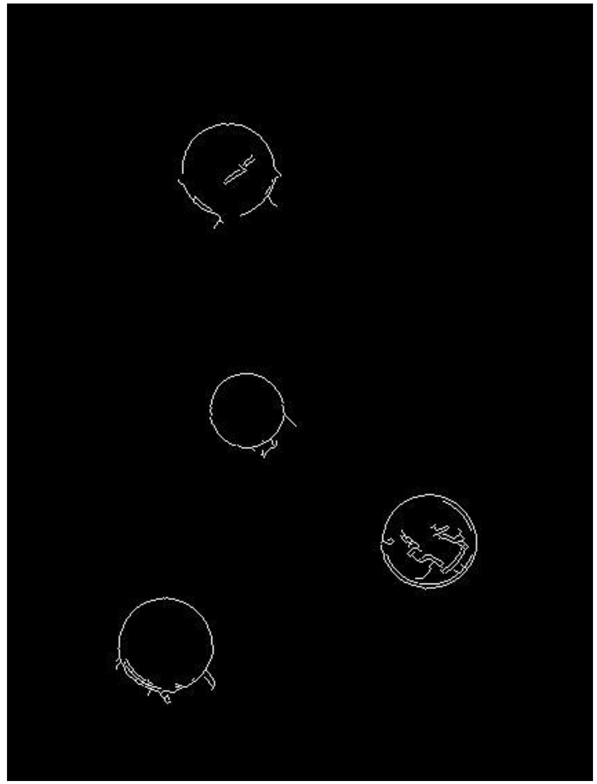
Example: detecting circles with Hough

Original

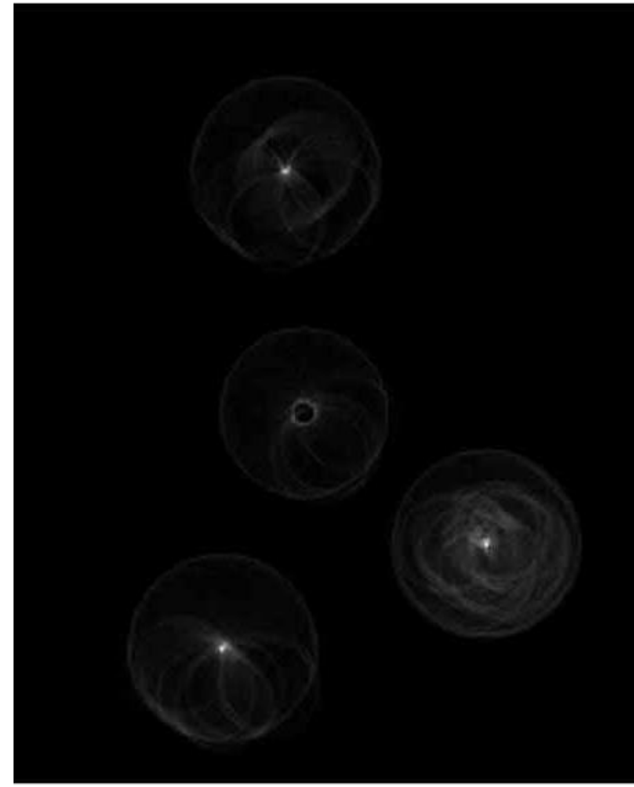
Combined detections



Edges

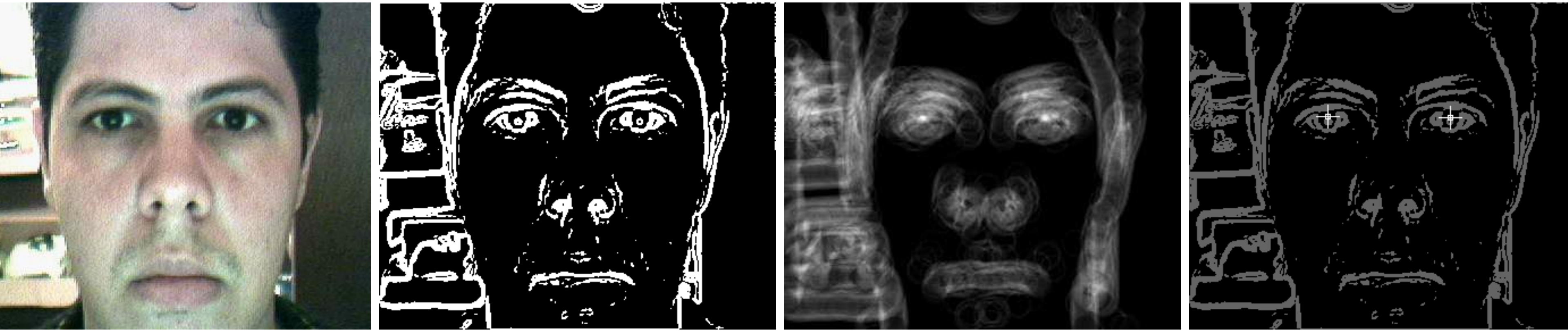


Votes: Quarter



Coin finding sample images from: Vivek Kwatra

Example: iris detection



Gradient+threshold | Hough space (fixed radius) | Max detections

- Hemerson Pistori and Eduardo Rocha Costa http://rsbweb.nih.gov/ij/plugins/hough-circles.html

Example: iris detection


Figure 2. Original image


Figure 3. Distance image


Figure 4. Detected face region

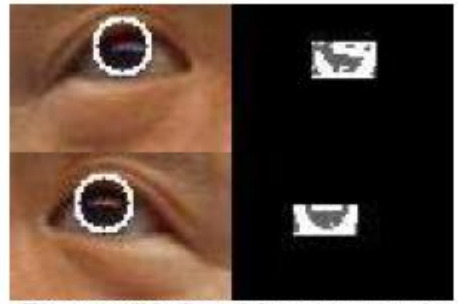


Figure 14. Looking upward

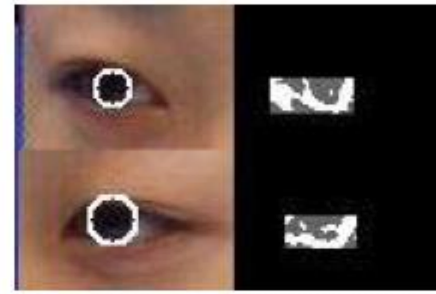


Figure 15. Looking sideways

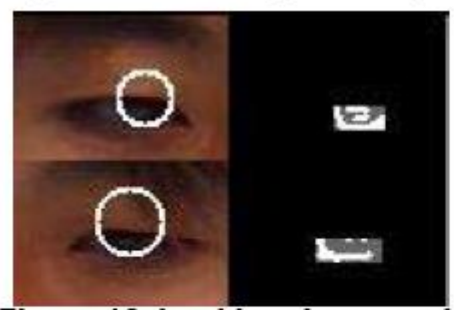


Figure 16. Looking downward

- An Iris Detection Method Using the Hough Transform and Its Evaluation for Facial and Eye Movement, by Hideki Kashima, Hitoshi Hongo, Kunihito Kato, Kazuhiko Yamamoto, ACCV 2002.

Voting: practical tips

- Minimize irrelevant tokens first
- Choose a good grid / discretization

  Too fine ← ? → Too coarse

- Vote for neighbors, also (smoothing in accumulator array)
- Use direction of edge to reduce parameters by 1
- To read back which points voted for “winning” peaks, keep tags on the votes.

Hough transform: pros and cons

Pros

- All points are processed independently, so can cope with occlusion, gaps
- Some robustness to noise: noise points unlikely to contribute *consistently* to any single bin
- Can detect multiple instances of a model in a single pass

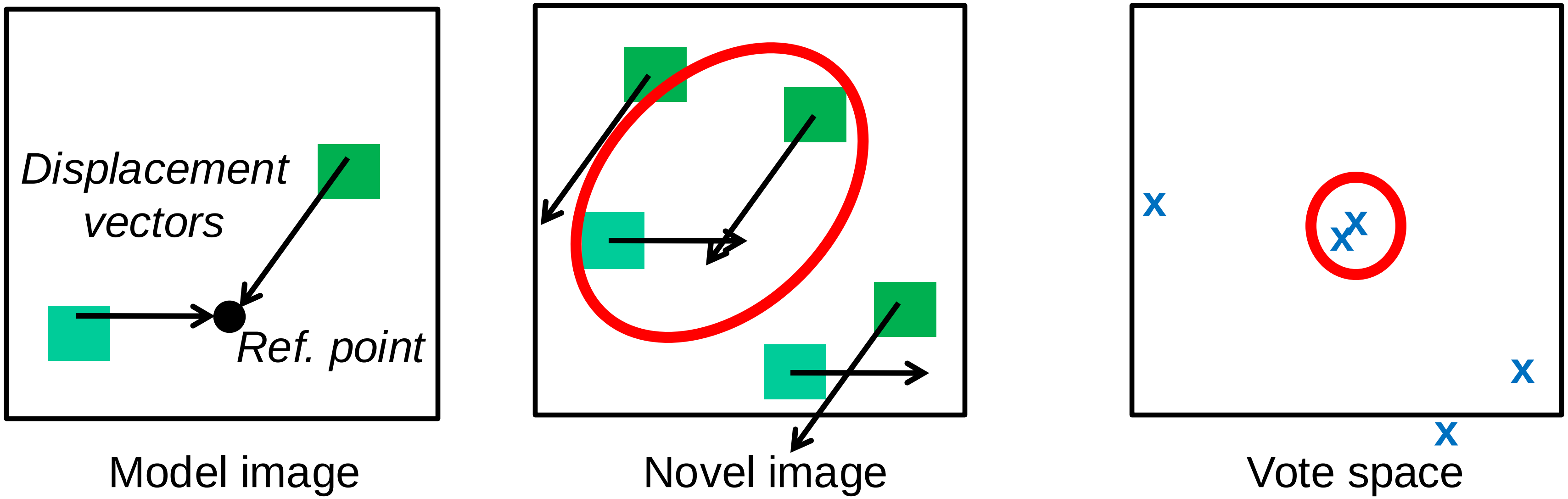
Cons

- Complexity of search time increases exponentially with the number of model parameters
- Non-target shapes can produce spurious peaks in parameter space
- Quantization: can be tricky to pick a good grid size

Generalized Hough Transform

- What if we want to detect arbitrary shapes?

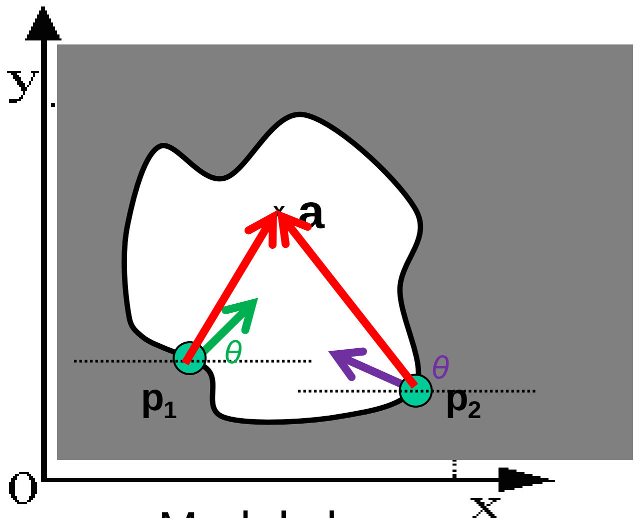
Intuition:



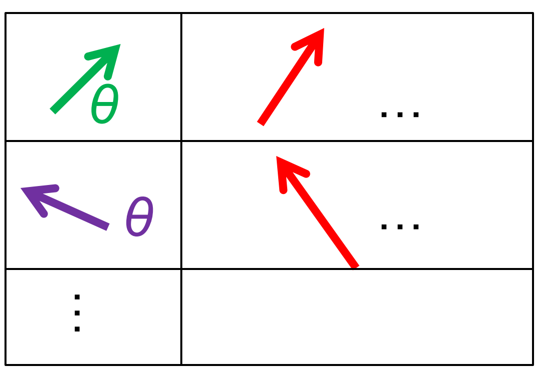
Now suppose those colors encode gradient directions…

Generalized Hough Transform

- Define a model shape by its boundary points and a reference point.



Model shape



<u>Offline procedure:</u>

At each boundary point, compute displacement vector: $\mathbf{r} = \mathbf{a} - \mathbf{p}_i$.

Store these vectors in a table indexed by gradient orientation θ.

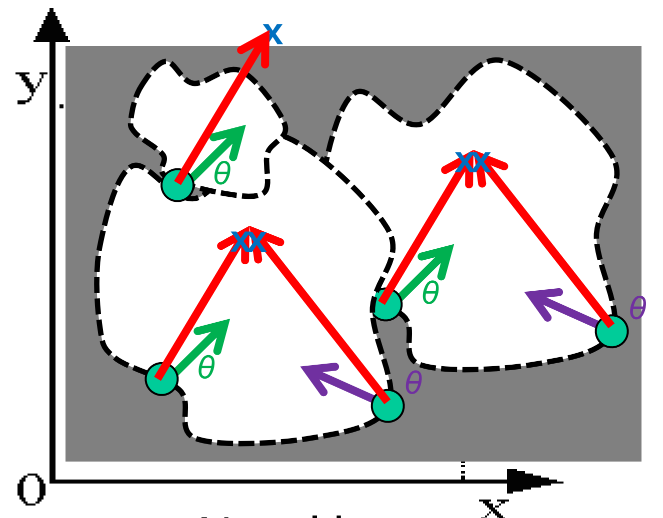
[Dana H. Ballard, Generalizing the Hough Transform to Detect Arbitrary Shapes, 1980]

Generalized Hough Transform

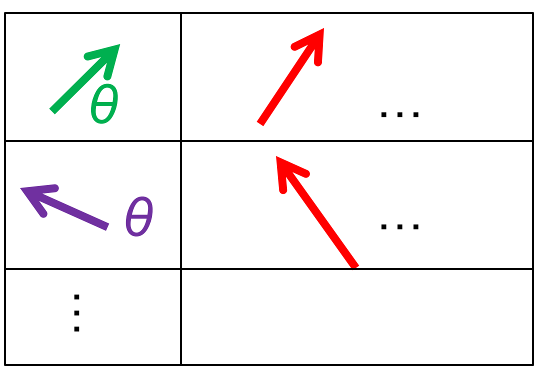
Detection procedure:

For each edge point:

- Use its gradient orientation θ to index into stored table
- Use retrieved **r** vectors to vote for reference point



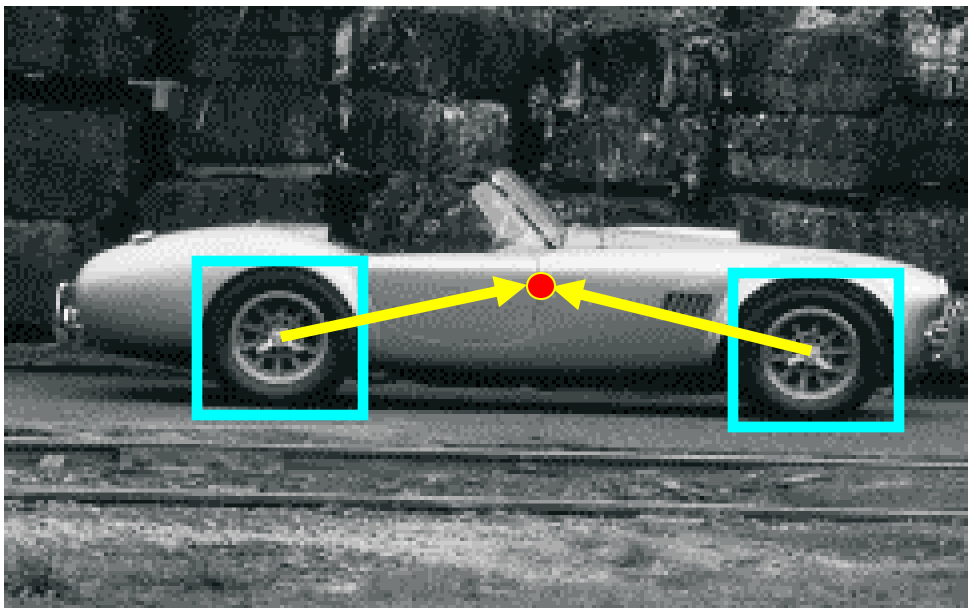
Novel image



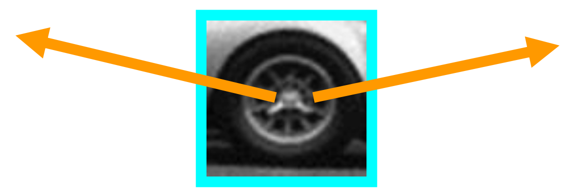
Assuming translation is the only transformation here, i.e., orientation and scale are fixed.

Generalized Hough for object detection

- Instead of indexing displacements by gradient orientation, index by matched local patterns.



training image

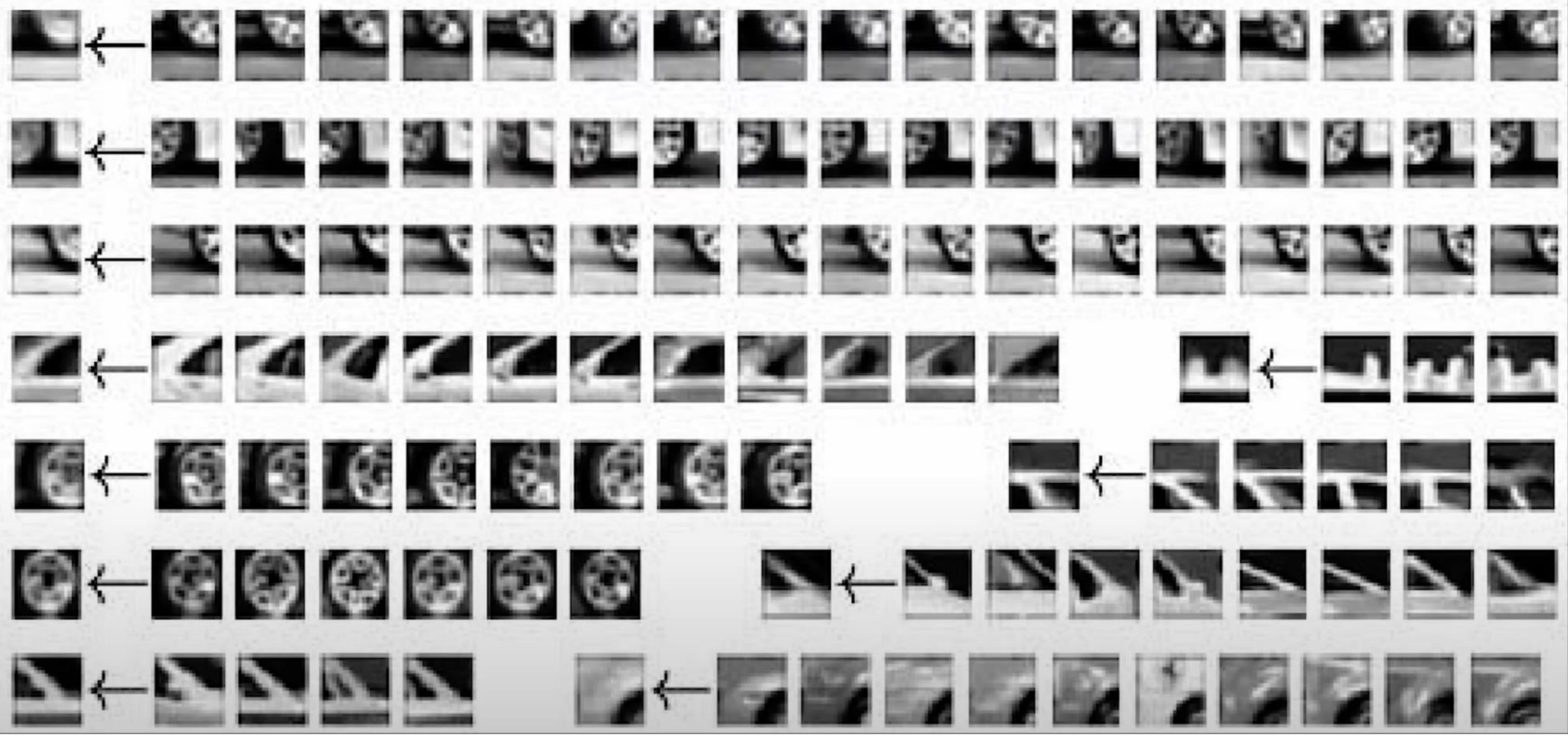


“visual codeword” with displacement vectors

B. Leibe, A. Leonardis, and B. Schiele, [Combined Object Categorization and Segmentation with an Implicit Shape Model](), ECCV Workshop on Statistical Learning in Computer Vision 2004

Source: L. Lazebnik

Visual code-words [Eg. of Clustering]



Generalized Hough for object detection

- Instead of indexing displacements by gradient orientation, index by “visual codeword”

test image

B. Leibe, A. Leonardis, and B. Schiele, Combined Object Categorization and Segmentation with an Implicit Shape Model, ECCV Workshop on Statistical Learning in Computer Vision 2004

Source: L. Lazebnik

Summary

- **Fitting** problems require finding any supporting evidence for a model, even within clutter and missing features.
  - associate features with an explicit model

- **Voting** approaches, such as the Hough transform, find likely model parameters without searching all combinations of features.
  - Hough transform approach for lines, circles, …, arbitrary shapes defined by a set of boundary points, recognition from patches.